

Today's Agenda:-

1. Range Divisibility
2. Alice & Capitalising.
3. SuperStream Engineers
4. Noble Integer — Same as assignments.

Problem Description

Samantha is a teacher who is teaching a group of students about divisibility by 7. To make the lesson more interesting, she gives them an array **A** of size **N** and asks them **Q** queries given by **B**.

In each query, she gives them a range from **L** to **R** given by **B[i][0]** and **B[i][1]** and asks them to find the count of numbers divisible by 7 in subarray **A[L], A[L+1], ... A[R]**. Since some students are struggling to solve the problem, Samantha asks for your help to come up with a solution.

Problem Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

$$1 \leq A[i] \leq 10^9$$

$$0 \leq B[i][0], B[i][1] < N$$

Input 1:

A = [2, 7, 14, 5, 7]
B = [[0, 2], [2, 4]]

Output 1:

[2, 2]

Input 2:

A = [7, 14, 2, 14]
B = [[1, 2], [0, 3]]

Output 2:

[1, 3]

 $O \times N$ 10^{10}

A: 0 1 2 3 4.
 { 2, 7, 14, 5, 7 }.

:- { 0, 1, 1, 0, 1 }.

pSum :- { 0, 1, 2, 2, 3 }.

$[L, R] = pSum[R] - pSum[L-1]$.

```
vector<int> Solution::solve(vector<int> &A, vector<vector<int> > &B) {

    //Modify the input array in-place to check for divisibility by 7 :- 1 if divisible, 0 if not divisible.
    for(int i=0; i<A.size(); i++) {
        if(A[i] % 7 == 0) {
            A[i] = 1;
        } else {
            A[i] = 0;
        }
    }

    //Compute the Prefix Sum in-place to handle range queries
    for(int i=1 ;i<A.size(); i++) {
        A[i] += A[i-1];
    }

    //Create a vector array of the size of queries
    vector<int>ans(B.size());

    //Iterate over all the queries and compute the numbers divisible by 7 in a range.
    for(int i=0; i<B.size(); i++) {
        int left = B[i][0];
        int right = B[i][1];

        if(left == 0) {
            ans[i] = A[right];
        } else {
            ans[i] = A[right] - A[left-1];
        }
    }

    //Return the Answer
    return ans;
}
```

Problem Description

Alice is developing a function that processes strings for a text analysis tool. She wants to write a function that capitalizes the segment of a **String A** starting from the first occurrence of a given **character B** to the end of the string. If the given character is not found, the string should remain unchanged.

For example, if the string is "**good morning**" and the given character is "**m**", then the function should capitalize the segment that starts from the first occurrence of "**m**" to the end of the string. The resulting string will be "good **MORNING**".

Complete the given function to help Alice accomplish this task.

Problem Constraints

- $1 \leq A.length \leq 10^5$
- $A[i]$ contains lowercase English Alphabets and spaces.
- $B.length = 1$
- B contains a lowercase English Alphabet

Example Input

Input 1:

A = "hello world"

B = "o"

hello WORLD

Input 2:

A = "coding is love"

B = "l"

coding is LOVE

Input 3:

A = "scaler academy"

B = "z"

Approach

1. Find index of first occurrence of B.

Example Output

Output 1:

"hello WORLD"

Output 2:

"coding is LOVE"

Output 3:

"scaler academy"

2. Iterate again on the string. For index $< i$, acquire as it is ; else, capitalise it & acquire.

```
public class Solution {
    public String capitalizeFromChar(String A, String B) {
        int index = -1;

        // Get the index of first occurrence of B[0]
        for(int i=0; i<A.length(); i++) {
            if(A.charAt(i) == B.charAt(0)) {
                index = i;
                break;
            }
        }

        // Handle Character not found
        if(index == -1) {
            return A;
        }

        char[] ans = new char[A.length()];

        // Iterate over each character
        for(int i=0; i<A.length(); i++) {

            // Keep as is the characters before the index
            if(i < index) {
                ans[i] = A.charAt(i);
            } else {

                // Capitalise lower case characters
                if(A.charAt(i) >= 97 && A.charAt(i) <= 122) {
                    ans[i] = (char) (A.charAt(i) - 32);
                } else {
                    ans[i] = A.charAt(i);
                }
            }
        }

        return new String(ans);
    }
}
```

Problem Description

You've just been hired as a network engineer at SuperStream, a leading video streaming service. One of your first tasks is to optimize the number of video data packets sent to users based on their internet connectivity.

When a user hits "play," video data is transmitted in packets. If their device acknowledges these packets quickly, it means they have a strong connection and can receive more packets simultaneously for smoother streaming. If acknowledgments lag, fewer packets should be sent to prevent buffering.

Given an array **A**, where each entry represents the acknowledgment time (in milliseconds) for individual packets, and two integers **B** and **C**, can you determine if there's a continuous sequence of **B** packets with an average acknowledgment time less than or equal to **C** milliseconds? If so, it's a green signal (integer **1**) to send more packets. Otherwise, it's time to throttle back (integer **0**).

Note: For average, take the **floor** of (sum/total number of elements).

Problem Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq A[i] \leq 10^9$$

$$1 \leq B \leq N$$

$$1 \leq C \leq 10^9$$

$$10^9, 10^9, 10^9 \dots 10^9$$

10^5 times

$$10^{14}$$

$$A: \underline{22, 15, 30, 26, 23, \dots 3}$$

$$B = 3$$

$$C = 20$$

$$(N-B+1) \times \underline{B}, \quad N^2$$

Approach:- Efficiently check the average of B consecutive packets using sliding window approach.

```
int Solution::solve(vector<int> &A, int B, int C) {
```

```
    //Take average of the original window
```

```
    long sum = 0;
```

```
    for(int i=0 ;i<B; i++) {
```

```
        sum = sum + A[i];
```

```
    }
```

```
    int avg = sum / B;
```

```
    // If the first window itself is having average <= C, return 1.
```

```
    // No further processing needed.
```

```
    if(avg <= C) {
```

```
        return 1;
```

```
    }
```

```
    // Slide over the remaining windows to calculate new averages.
```

```
    // Anytime <= C is encountered return 1
```

```
    for(int i = B; i < A.size(); i++) {
```

```
        sum = sum + A[i] - A[i-B];
```

```
        avg = sum / B;
```

```
        if(avg <= C) {
```

```
            return 1;
```

```
        }
```

```
    }
```

```
    // If function didn't return from for loop, it means
```

```
    // it never encountered valid avg thus return 0 at this point.
```

```
    return 0;
```

```
}
```

Q. Noble Integers

Discussed in Class

```
int Solution::solve(vector<int> &A) {
    sort(A.begin(), A.end(), greater<int>());
    int count = 0;
    if(A[0] == 0)
        return 1;

    for(int i = 1; i < A.size(); i++){
        if(A[i] != A[i-1]){
            count = i;
        }

        if(count == A[i])
            return 1;
    }
    return -1;
}
```

Reattempts:-

R_1 :- $T+1$ to $T+8^{\text{th}}$ day
15th May 21st May
2nd 28th May
 R_2 :- $T+9$ to $T+15^{\text{th}}$ day
2nd 4th June
 R_3 :- $T+16$ to $T+22^{\text{nd}}$ day.

50 Q:-









